

---

Oliver Krauss

# Modern Code - Strings & File Access

ModernCode | MTD.ba VZ SS25

Hagenberg 25.11.2025

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY  
OF APPLIED SCIENCES  
UPPER AUSTRIA

# Strings & File Access: Why It Matters

- **The Problem:** Programs need to process text and persist data
- **The Solution:** Strings for text manipulation, files for data storage
- **Why It Matters:** Almost every real program works with text and files
- **Real-world examples:**
  - Game: Save player progress, load configuration files, display dialogue
  - Web: Process user input, validate forms, read/write databases
  - Data Science: Parse CSV files, clean text data, analyze logs

# Connection to Previous Topics

- **Arrays**: Strings are essentially arrays of characters
- **Loops**: Iterate through characters in strings
- **Methods**: Break down string processing into reusable functions
- **Control Structures**: Validate input, check conditions
- **New Concept**: Working with external data (files)

# String Fundamentals

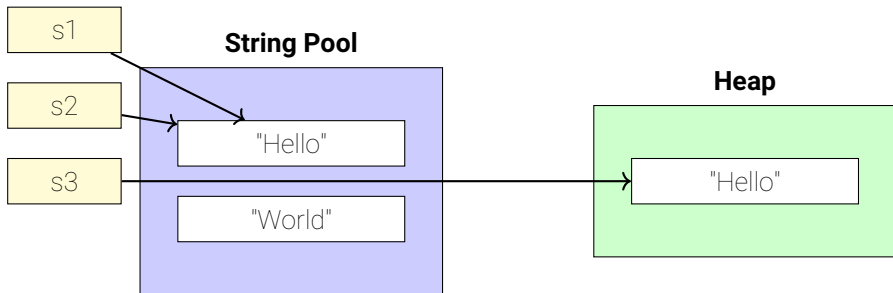
```
1 String example = "Hello World";
```

- **What is a String?** A sequence of characters
- **Key Property:** Strings are **immutable** (cannot be changed)
- **Memory Model:** String objects live on the heap
- **String Pool:** Java optimizes string storage with a special pool

# String Creation: Literal vs new String()

```
1  // String literal (preferred)
2  String s1 = "Hello";
3  String s2 = "Hello";
4
5  // Using new keyword (not recommended)
6  String s3 = new String("Hello");
7
8  IO.println(s1 == s2);  // true
9  IO.println(s1 == s3);  // false -> WAIT WHAT?
```

# String Pool: Memory Optimization



**Key Point:** String literals share the same object, `new String()` creates separate objects

# Understanding == and .equals()

## Two Different Types of Comparison:

### -- == (Reference Comparison):

- For **objects**: Compares memory addresses (where objects are stored)
- For **primitives**: Compares actual values (int, char, boolean)
- Asks: "Are these the SAME object?" (objects) or "Same value?" (primitives)
- **Problem**: Two different String objects with same content are **false**

### -- .equals() (Content Comparison):

- Compares the **actual content** of objects
- Asks: "Do these objects have the SAME content?"
- Method defined in Object class, overridden by String
- **Solution**: Always use this for String comparison!

**Key Rule:** For strings, use `.equals()`, not `==`

# String Immutability: What It Means

```
1  String name = "Alice";  
2  name = name + " Smith";  // Creates NEW string
```

## What actually happens:

1. "Alice" is created
2. "Alice Smith" is created as a NEW string
3. `name` now points to "Alice Smith"
4. "Alice" still exists (if no other references, garbage collected)

**Why immutable?** Thread safety, security, performance optimization



# String Comparison: Using == (DON'T DO THIS!)

```
1  String s1 = "Hello";
2  String s2 = "Hello";
3  String s3 = new String("Hello");
4  String s4 = "Hel" + "lo"; // Compile-time concatenation
5
6  // == compares REFERENCES (memory addresses)
7  IO.println(s1 == s2); // true (same object in pool)
8  IO.println(s1 == s3); // false (different objects)
9  IO.println(s1 == s4); // true (compile-time optimization)
```

**NEVER use == for String comparison!**

# GOOD String Comparison: Using .equals()

```
1  String s1 = "Hello";
2  String s2 = "Hello";
3  String s3 = new String("Hello");
4  String s4 = "Hel" + "lo"; // Compile-time concatenation
5
6  // .equals() compares CONTENT
7  IO.println(s1.equals(s2)); // true
8  IO.println(s1.equals(s3)); // true
9  IO.println(s1.equals(s4)); // true
```

**Always use .equals() to compare string content!**

# Basic String Operations

- **Access:** Get individual characters or substrings
- **Search:** Find characters or substrings
- **Transform:** Change case, trim whitespace
- **Check:** Test properties of strings

# Length and Character Access

```
1  String text = "Hello, World!";
2  // Get length
3  int len = text.length(); // 13
4
5  // Access individual characters (0-indexed)
6  char first = text.charAt(0); // 'H'
7  char last = text.charAt(len - 1); // '!'
8  // Iterate through characters
9  for (int i = 0; i < text.length(); i++) {
10     char c = text.charAt(i);
11     IO.println(c);
12 }
```

# String Methods vs Array Properties

## Crucial Difference:

- **String methods** (objects): Use **parentheses ()**
  - `text.length()` - method call
  - `text.charAt(0)` - method call
  - `text.equals(other)` - method call
- **Array properties** (primitives): **No parentheses**
  - `array.length` - property access
  - `array[0]` - index access

**Remember: Strings are objects, arrays are primitive structures!**

# Searching in Strings

```
1  String text = "Hello, World!";
2
3  // Find first occurrence
4  int pos1 = text.indexOf('o');      // 4
5  int pos2 = text.indexOf("World");  // 7
6  int pos3 = text.indexOf('x');      // -1 (not found)
7  // Find last occurrence
8  int pos4 = text.lastIndexOf('o');  // 8
9  // Check if string contains substring
10 boolean hasWorld = text.contains("World"); // true
```

All index methods return -1 if not found, otherwise returns position

# Character vs String: Single vs Double Quotes

```
1 // Single quotes ' ' = CHARACTER (primitive type)
2 char letter = 'A';           // Single character
3
4 // Double quotes " " = STRING (object type)
5 String word = "Hello";       // Sequence of characters
```

## Key Differences:

- **Type:** char vs String
- **Size:** Single character vs sequence of characters
- **Memory:** Primitive vs object
- **Methods:** No methods vs many methods

**Never mix them up: 'A' ≠ "A"**

# Extracting Substrings

```
1  String text = "Hello, World!";
2
3  // substring(start) - from start to end
4  String part1 = text.substring(7);      // "World!"
5
6  // substring(start, end) - from start to end (exclusive)
7  String part2 = text.substring(0, 5);   // "Hello"
8  String part3 = text.substring(7, 12);  // "World"
```



# Case Conversion and Trimming

```
1  String text = "  Hello, World!  ";
2
3  // Case conversion (creates NEW strings)
4  String upper = text.toUpperCase(); // "  HELLO, WORLD!  "
5  String lower = text.toLowerCase(); // "  hello, world!  "
6  // Remove leading/trailing whitespace
7  String trimmed = text.trim();      // "Hello, World!"
8  // Check case
9  boolean sU = Character.isUpperCase(text.charAt(2)); // true
10 // Case-insensitive comparison
11 boolean same = "hello".equalsIgnoreCase("HELLO"); // true
```

# String Checking Methods

```
1  String text = "Hello123";
2
3  // Check if empty
4  boolean empty1 = text.isEmpty();           // false
5  boolean empty2 = "".isEmpty();             // true
6
7  // Check start/end
8  boolean startsH = text.startsWith("Hello"); // true
9  boolean ends123 = text.endsWith("123");    // true
```

# Advanced String Operations

- **Splitting**: Break strings into parts
- **Replacing**: Substitute characters or substrings
- **Formatting**: Create formatted strings
- **StringBuilder**: Efficiently build strings

# Splitting Strings

```
1 String csv = "Alice,25,Engineer";  
2 String[] parts = csv.split(",");  
3 // Result: ["Alice", "25", "Engineer"]
```

## How it works:

- `split()` breaks a string into an array
- Takes a delimiter (what to split on) as parameter
- Returns **String[]** - array of strings
- Original string is unchanged (immutability)

**Common use case:** Processing CSV data, parsing user input

# Replacing in Strings

```
1 String text = "Hello World World";
2 String result = text.replace("World", "Java");
3 // Result: "Hello Java Java"
4 String result = text.replace('o', 'X');
5 // Result: "HellX WXrld WXrld"
```

## How it works:

- **replace()** finds and replaces text
- Takes two parameters: old text, new text
- Replaces **ALL** occurrences
- Returns **new string** (original unchanged)

**Common use case:** Data cleaning, text processing

# String Formatting I

```
1  String name = "Alice";
2  int age = 25;
3  double score = 95.5;
4
5  String formatted = String.format(
6      "Name: %s, Age: %d, Score: %.2f",
7      name, age, score
8  );
9  // Result: "Name: Alice, Age: 25, Score: 95.50"
```

# String Formatting II

## Format Specifiers:

- **%s** - inserts a string
- **%d** - inserts an integer
- **%f** - inserts a floating-point number
- **%.2f** - floating-point with 2 decimal places
- **%n** - platform-independent newline

**Why use this?** Clean, readable output formatting

# String Concatenation: The Problem

```
1  String result = "";
2  for (int i = 0; i < 1000; i++) {
3      result = result + i + ",";
4  }
5  // Creates 1000 intermediate string objects!
```

## Why is this slow?

- Each + creates a **new string**
- Copies all previous characters
- Adds new characters
- Old strings become garbage

**Problem:** String concatenation in loops is very inefficient



# StringBuilder: The Solution

```
1  StringBuilder sb = new StringBuilder();
2  for (int i = 0; i < 1000; i++) {
3      sb.append(i).append(",");
4  }
5  String result = sb.toString();
```

## How it works:

- **StringBuilder** is **mutable** (can be changed)
- **append()** adds to existing buffer (no copying)
- **toString()** creates final string when done
- Only creates **one** string object total

**Rule:** Use **StringBuilder** for string building in loops

# StringBuilder Methods

```
1  StringBuilder sb = new StringBuilder("Hello");
2  sb.append(" World");           // "Hello World"
3  sb.insert(5, " Java");         // "Hello Java World"
4  sb.delete(5, 10);             // "Hello World"
5  sb.reverse();                 // "dlroW olleH"
6  sb.setCharAt(0, 'X');         // "XlroW olleH"
```

## Key Methods:

- **append()** - add to end
- **insert()** - add at specific position
- **delete()** - remove characters
- **reverse()** - reverse the content
- **setCharAt()** - change single character

# StringBuilder Example: CSV Builder

```
1  String[] names = {"Alice", "Bob", "Charlie"};
2  int[] scores = {95, 87, 92};
3
4  StringBuilder csv = new StringBuilder();
5  csv.append("Name,Score\n"); // Header
6
7  for (int i = 0; i < names.length; i++) {
8      csv.append(names[i])
9          .append(",")
10         .append(scores[i])
11         .append("\n");
12 }
```

# String vs StringBuilder: Performance

| Operation     | String                     | StringBuilder             |
|---------------|----------------------------|---------------------------|
| Concatenation | Slow (creates new objects) | Fast (modifies buffer)    |
| Memory        | High (many temp objects)   | Low (single buffer)       |
| Mutability    | Immutable                  | Mutable                   |
| Thread Safety | Safe                       | Not safe                  |
| Use Case      | Small, fixed strings       | Building strings in loops |

**Rule of Thumb:** Use String for simple operations, StringBuilder for complex building

# Regular Expressions: Pattern Matching

- **What is regex?** A language for describing text patterns
- **Use cases:** Validation, searching, replacing, parsing
- **Examples:** Email validation, phone number formatting, data extraction
- **Trade-off:** Powerful but can be complex
- **Dialects:** Perl, Python, Java, JavaScript, PHP, ...all have similar but not identical syntax

# Basic Regex Syntax

| Pattern | Meaning              | Example                                 |
|---------|----------------------|-----------------------------------------|
| .       | Any single character | <b>a.c</b> matches "abc", "a5c"         |
| *       | Zero or more         | <b>ab*c</b> matches "ac", "abc", "abbc" |
| +       | One or more          | <b>ab+c</b> matches "abc", "abbc"       |
| ?       | Zero or one          | <b>ab?c</b> matches "ac", "abc"         |
| [abc]   | Any of a, b, c       | <b>[aeiou]</b> matches vowels           |
| [0-9]   | Any digit            | <b>[0-9]+</b> matches numbers           |
| ^       | Start of string      | <b>^Hello</b> matches start             |
| \$      | End of string        | <b>end\$</b> matches end                |

# Common Regex Patterns

| Pattern            | Matches                               |
|--------------------|---------------------------------------|
| <code>\d</code>    | Any digit (0-9)                       |
| <code>\D</code>    | Any non-digit                         |
| <code>\w</code>    | Any word character (a-z, A-Z, 0-9, _) |
| <code>\W</code>    | Any non-word character                |
| <code>\s</code>    | Any whitespace (space, tab, newline)  |
| <code>\S</code>    | Any non-whitespace                    |
| <code>{n}</code>   | Exactly n times                       |
| <code>{n,m}</code> | Between n and m times                 |

# Using Regex in Java: matches()

```
1  // Simple pattern matching with matches()
2  String text1 = "12345";
3  boolean isDigits = text1.matches("\\d+"); // true
4
5  String text2 = "Hello123";
6  boolean isLetters = text2.matches("[a-zA-Z]+"); // false
7
8  // Phone number pattern (simple)
9  String phone = "555-1234";
10 boolean validPhone = phone.matches("\\d{3}-\\d{4}"); // true
```

**Note:** In Java strings, backslash must be escaped: `\\d`



# replaceFirst() and replaceAll()

```
1  String text = "Hello123 World456 Java789";
2
3  // Replace first match only
4  String result1 = text.replaceFirst("\\d+", "X");
5  // Result: "HelloX World456 Java789"
6  // Replace all matches
7  String result2 = text.replaceAll("\\d+", "X");
8  // Result: "HelloX WorldX JavaX"
9  // Replace with different patterns
10 String result3 = text.replaceAll("[a-z]+", "WORD");
11 // Result: "WORD123 WORD456 WORD789"
```

# AI Support: Understanding Regex

**Regex can be complex! Use AI to help:**

**Good prompts for AI:**

- "Explain this regex pattern: `^[\w.]+@[\w-]+\.[\w]{2,6}$`"
- "Create a regex pattern to match dates in format DD/MM/YYYY"
- "What does this regex do: `\d{3}-\d{3}-\d{4}`"
- "Debug this regex: why doesn't it match phone numbers?"

**AI can help with:**

- Explaining complex patterns
- Generating patterns for specific use cases
- Debugging regex that don't work as expected
- Suggesting test cases

# File I/O: Introduction

- **Goal:** Read data from files and write data to files
- **Why files?** Persist data beyond program execution
- **Common uses:** Save game progress, load configuration, process data
- **Key concepts:** Streams, readers/writers, resource management

# File Class: Working with Files

## **File class provides file system operations:**

- Check if file exists
- Get file properties (size, path, name)
- Create, delete, rename files
- Check if path is file or directory

**Important:** File object represents a path, not the file content

# File Class: Creating and Checking

```
1  import java.io.File;  
2  
3  File file = new File("data.txt");  
4  boolean exists = file.exists();
```

## Key concepts:

- **File** object represents a file path (not the file itself)
- **exists()** checks if file actually exists on disk
- File path can be relative or absolute

**Important:** Creating a **File** object doesn't create the actual file!

# File Class: Getting Information

```
1  File file = new File("data.txt");  
2  
3  String name = file.getName();           // "data.txt"  
4  String path = file.getAbsolutePath();   // full path  
5  long size = file.length();              // size in bytes
```

## Common methods:

- `getName()` - just the filename
- `getAbsolutePath()` - complete path
- `length()` - file size in bytes

# File Class: File Operations

```
1  File file = new File("data.txt");  
2  
3  boolean isFile = file.isFile();  
4  boolean isDir = file.isDirectory();  
5  boolean created = file.createNewFile();  
6  boolean deleted = file.delete();
```

## Operations:

- `isFile()` / `isDirectory()` - check type
- `createNewFile()` - create if doesn't exist
- `delete()` - remove file

**Note:** Always check if operations succeeded!

# Exception Handling for File I/O

## File operations can fail:

- **FileNotFoundException**: File doesn't exist
- **IOException**: General I/O error (permissions, disk full, etc.)
- **Must handle**: Java requires exception handling for file I/O

## Two approaches:

- **try-catch**: Handle exceptions in the method
- **throws**: Declare that method throws exceptions



# Using throws: Let Someone Else Handle It

```
1  public static void readFile(String filename)
2      throws FileNotFoundException {
3
4      File file = new File(filename);
5      ...
6  }
```

## How it works:

- Method declares it might throw an exception with the **throws** keyword
- Caller must handle the exception
- Useful for utility methods

# try-catch-finally Syntax

```
1  try {  
2      ...  
3  } catch (FileNotFoundException e) {  
4      IO.println("File not found!");  
5  } catch (IOException e) {  
6      IO.println("I/O error occurred!");  
7  } finally {  
8      // cleanup code  
9  }
```

**try block:** Contains risky code that might throw exceptions

**catch block:** Handles specific exception types

**multiple catches:** Handle different exception types

**finally block:** Always runs (success or failure)

# try-with-resources Syntax

```
1  try (Scanner s = new Scanner(file)) {  
2      ...  
3  } catch (FileNotFoundException e) {  
4      IO.println("File not found!");  
5  }
```

**resource declaration:** Create resource in try parentheses

**exception handling:** Same catch blocks as before

**auto cleanup:** Resource automatically closed here

## Example: Reading Files with try-catch

```
1  try {
2      File file = new File(filename);
3      Scanner scanner = new Scanner(file);
4      while (scanner.hasNextLine()) {
5          String line = scanner.nextLine();
6          IO.println(line);
7      }
8      scanner.close();
9  } catch (FileNotFoundException e) {
10     IO.println("File not found: " + filename);
11 }
```

## Example: Reading Files with try-with-resources

```
1  try (Scanner scanner = new Scanner(new File(filename))) {  
2      while (scanner.hasNextLine()) {  
3          String line = scanner.nextLine();  
4          IO.println(line);  
5      }  
6      // scanner automatically closed here!  
7  } catch (FileNotFoundException e) {  
8      IO.println("File not found: " + filename);  
9  }
```

- No manual `scanner.close()`
- Automatic cleanup even if exception occurs
- Cleaner, less error-prone code

# Reading Files: Different Approaches

## Sorted by complexity (simple to advanced):

1. **Files.readAllLines()**: Read entire file at once (simplest)
2. **Scanner**: Easy to use, good for text files
3. **BufferedReader**: More efficient, better for large files

## Choose based on:

- File size
- Processing needs
- Performance requirements

## Files.readAllLines(): Simplest Approach

```
1  try {  
2      List<String> lines = Files.readAllLines(Paths.get("data.txt"));  
3  
4      for (String line : lines) {  
5          IO.println(line);  
6      }  
7  } catch (IOException e) {  
8      IO.println("Error reading file: " + e.getMessage());  
9  }
```

- One line to read entire file
- Returns List<String> of all lines
- No manual resource management

# Reading with Scanner

```
1  try (Scanner scanner = new Scanner(new File("data.txt"))) {  
2      while (scanner.hasNextLine()) {  
3          String line = scanner.nextLine();  
4          IO.println(line);  
5      }  
6  } catch (FileNotFoundException e) {  
7      IO.println("File not found: " + e.getMessage());  
8  }
```

**Good for:** Simple text files, line-by-line processing



# Scanner: Reading Different Types

```
1  // File content: "Alice 25 95.5"
2  try (Scanner scanner = new Scanner(new File("data.txt"))) {
3      String name = scanner.next();      // "Alice"
4      int age = scanner.nextInt();       // 25
5      double score = scanner.nextDouble(); // 95.5
6
7      IO.println("Name: " + name);
8      IO.println("Age: " + age);
9      IO.println("Score: " + score);
10 } catch (FileNotFoundException e) {
11     IO.println("Error: " + e.getMessage());
12 }
```

# Reading with BufferedReader

```
1  try (BufferedReader reader = new BufferedReader(  
2      new FileReader("data.txt"))) {  
3      String line;  
4      while ((line = reader.readLine()) != null) {  
5          IO.println(line);  
6      }  
7  } catch (IOException e) {  
8      IO.println("Error: " + e.getMessage());  
9  }
```

**Good for:** Large files, better performance than Scanner

# Writing Files: Different Approaches

- **FileWriter**: Simple text writing
- **BufferedWriter**: Efficient writing with buffering
- **PrintWriter**: Convenient formatted output

## Write modes:

- **Overwrite**: Replace existing file content (default)
- **Append**: Add to end of existing file

# Line Endings: Cross-Platform Problem

## Different operating systems use different line endings:

- **Windows:** CRLF (`\r\n`)
- **Unix/Linux/Mac:** LF (`\n`)
- **Old Mac:** CR (`\r`)

## Problem with using one (ex. `\n`):

- Hard-coded to OS-specific line endings
- Files may not display correctly on other OS
- Not portable across different systems

**Solution:** Use `System.lineSeparator()` (Uses the executing PC's OS's line endings)

# Writing with FileWriter

```
1  try (FileWriter writer = new FileWriter("output.txt")) {  
2      writer.write("Hello, World!" + System.lineSeparator());  
3      writer.write("This is line 2" + System.lineSeparator());  
4      writer.write("And this is line 3" + System.lineSeparator());  
5      IO.println("File written successfully");  
6  } catch (IOException e) {  
7      IO.println("Error: " + e.getMessage());  
8  }
```

# Append Mode

```
1  // Overwrite mode (default)
2  try (FileWriter writer = new FileWriter("output.txt")) {
3      writer.write("This replaces file content" + System.
4          lineSeparator());
5  }
6  // Append mode (second parameter = true)
7  try (FileWriter writer = new FileWriter("output.txt", true)) {
8      writer.write("This is added to the end" + System.lineSeparator
9          ());
10 } catch (IOException e) {
11     IO.println("Error: " + e.getMessage());
12 }
```

# Writing with BufferedWriter

```
1  try (BufferedWriter writer = new BufferedWriter(  
2      new FileWriter("output.txt"))) {  
3      writer.write("Line 1");  
4      writer.newLine(); // Platform-independent newline  
5      writer.write("Line 2");  
6      writer.newLine();  
7      ...  
8  } catch (IOException e) {  
9      IO.println("Error: " + e.getMessage());  
10 }
```

# Writing with PrintWriter

```
1  try (PrintWriter writer = new PrintWriter("output.txt")) {
2      writer.println("Hello, World!");
3      writer.println("This is line 2");
4      writer.printf("Score: %d, Grade: %s%n", 95, "A");
5
6      // Can also use print() without newline
7      writer.print("No newline");
8  } catch (IOException e) {
9      IO.println("Error: " + e.getMessage());
10 }
```

**Most convenient** `println()`, `printf()` like console output



# Common Mistakes: Overview

## **String mistakes:**

- Using == instead of .equals()
- String concatenation in loops
- Not handling null strings

## **File I/O mistakes:**

- Not closing resources
- Not handling exceptions
- Wrong file paths
- Not checking if file exists

# Mistake: String Comparison with ==

```
1  // WRONG: Compares references, not content
2  String input = scanner.nextLine();
3  if (input == "yes") { // Usually false!
4      IO.println("Yes!");
5  }
6  // CORRECT: Compare content
7  if (input.equals("yes")) {
8      IO.println("Yes!");
9  }
10 // BEST: Case-insensitive
11 if (input.equalsIgnoreCase("yes")) {
12     IO.println("Yes!");
13 }
```

# Mistake: Not Closing Resources I

```
1  // WRONG: Resource leak
2  Scanner scanner = new Scanner(new File("data.txt"));
3  String line = scanner.nextLine();
4  // Forgot to close! File handle remains open
```

# Mistake: Not Closing Resources II

```
1  // CORRECT: Manual close
2  Scanner scanner = new Scanner(new File("data.txt"));
3  try {
4      String line = scanner.nextLine();
5  } finally {
6      scanner.close();
7  }
8
9  // BEST: try-with-resources
10 try (Scanner scanner = new Scanner(new File("data.txt"))) {
11     String line = scanner.nextLine();
12 }
```

# Mistake: Not Handling Exceptions

```
1  // WRONG: Won't compile (checked exception)
2  Scanner scanner = new Scanner(new File("data.txt"));
3
4  // CORRECT: Handle exception
5  try {
6      Scanner scanner = new Scanner(new File("data.txt")); ...
7  } catch (FileNotFoundException e) {
8      IO.println("File not found!");
9  }
10 // ALSO CORRECT: Declare throws
11 public static void readFile() throws FileNotFoundException {
12     Scanner scanner = new Scanner(new File("data.txt")); ...
13 }
```

# Mistake: Wrong File Paths

```
1  // WRONG: Absolute path (not portable)
2  File file = new File("C:\\Users\\Alice\\data.txt"); // Windows
    only!
3
4  // BETTER: Portable path construction
5  Path path = Paths.get("C:", "Users", "Alice", "data.txt");
6  File file = path.toFile();
```

# Completely Platform-Independent Paths

```
1 // BEST: Use system properties for user directory
2 String userHome = System.getProperty("user.home");
3 Path path = Paths.get(userHome, "data.txt");
4 File file = path.toFile();
```

## System Properties:

- `user.home` - User's home directory
- `user.dir` - Current working directory
- `file.separator` - Path separator for OS

# Mistake: Not Flushing Writers I

```
1  // PROBLEM: Data might not be written immediately
2  BufferedWriter writer = new BufferedWriter(
3      new FileWriter("output.txt"));
4  writer.write("Important data");
5  // If program crashes here, data might be lost!
```



# Mistake: Not Flushing Writers II

```
1  // SOLUTION 1: Flush manually
2  writer.flush(); // Force write to disk
3  // SOLUTION 2: Close (also flushes)
4  writer.close();
5
6  // BEST: try-with-resources (auto-closes)
7  try (BufferedWriter writer = new BufferedWriter(
8      new FileWriter("output.txt"))) {
9      writer.write("Important data");
10 } // Automatically flushed and closed
```

# Summary: Strings & File Access

## String Fundamentals

- Immutability and String pool
- `==` vs `.equals()` comparison
- Basic operations (`charAt`, `indexOf`, `substring`)

## Advanced Strings

- `split()`, `replace()`, `format()`
- `StringBuilder` for efficient building
- Regular expressions for pattern matching

## File Reading

- `Files.readAllLines()` for simple cases
- `Scanner` for line-by-line processing
- `BufferedReader` for large files
- `try-with-resources` for cleanup

## File Writing

- `FileWriter` for basic writing
- `BufferedWriter` for efficiency
- `PrintWriter` for formatted output
- Platform-independent line endings

## Exception Handling

- `try-catch-finally` syntax
- `try-with-resources` for cleanup
- `throws` vs `try-catch` approaches

## Key Principles

- Always close resources
- Handle exceptions properly
- Use `StringBuilder` in loops
- Use `.equals()` for string comparison